Software Test Plan (STP) — IEEE 829

Jackfruit — Comprehensive Verification Strategy & Baseline Test Suite

UE24CS341A — Software Engineering | Release 1.0 (Sprint 1)

Software Test Plan (STP) — Jackfruit Compression System

Project: Jackfruit File Compression & Decompression Tool
Version: 1.0
Author: Tanmayi (Testing & Coverage Lead)
Date: October 2026
Status: Approved

---

1. Introduction

- **Purpose:** This document defines the testing strategy, verification framework, test environment, and quality thresholds for the Jackfruit Huffman Compression system.
- **Scope:** Covers unit, integration, and end-to-end system testing for core modules: bitwise I/O, canonical/min-heap Huffman tree reconstruction, binary archive parsing, decompression byte restoration, CRC-32 integrity validation, and CLI execution.
- **References:** UE24CS341A Course Guidelines, Jackfruit SRS v0.1, Jackfruit Architecture SAD v0.1, Binary Archive Format Specification.
- **Definitions:** STP (Software Test Plan), SRS (Software Requirements Specification), RTM (Requirements Traceability Matrix), MSB (Most Significant Bit), CRC (Cyclic Redundancy Check).

2. Test Items

- BitReader / BitWriter bitwise streaming modules
- Frequency table generator & Huffman tree serializer/deserializer
- Archive header parsing & validation module (ArchiveReader)
- Decompression engine (bitstream parsing -> byte stream reconstruction)
- CRC-32 IEEE 802.3 checksum calculation and payload verifier
- Interactive CLI interface

3. Features to be Tested

Mapped directly to project requirements: * JACK-F-001 to JACK-F-004: File compression, tree generation, and archive creation * JACK-F-005: Archive header parsing and magic byte verification * JACK-F-006: Huffman tree reconstruction from serialized frequency/code data * JACK-F-007: BitReader sequential reading up to exact total_bits boundary * JACK-F-008: Byte-identical file restoration verified via hash comparison * JACK-F-009 to JACK-F-012: Archive statistics extraction and metrics calculation * JACK-F-013 to JACK-F-016: Archive integrity checks and CRC-32 mismatch detection * JACK-NF-001: Sub-second execution for standard payloads (< 10 MB)

4. Features Not to be Tested

- Third-party OS filesystem kernel drivers
- Third-party compression schemes (DEFLATE / LZMA / BZIP2)

- Hardware disk failures or physical media corruption

5. Test Approach / Strategy

- **Unit Testing:** Module-level isolated logic written using GoogleTest (gtest).
- **Integration Testing:** Interaction between BitReader, ArchiveReader, and the Huffman decoding tree.
- **System Testing:** Black-box end-to-end tests comparing original files against decompressed copies (cmp / SHA-256 byte-level verification) on empty, text, binary, and large inputs.
- **Coverage Standards:** Minimum 80% line coverage and 70% branch coverage collected via gcov/lcov.
- **Security & Robustness:** Address Sanitizer (ASan) and Undefined Behavior Sanitizer (UBSan) enabled on all automated CI test runs.
- **Entry Criteria:** Clean compilation with zero compiler warnings (-Wall -Wextra -Werror).
- **Exit Criteria:** 100% automated test pass rate, target test coverage achieved, and 0 memory leaks.

5.1 Security & Robustness Validation

- Fuzz testing against malformed and truncated archive headers.
- Verification of safe error exits upon CRC-32 checksum failures without buffer overflows.
- Path traversal sanitization on output filenames.

6. Test Environment

- **Platform:** Linux (Ubuntu 22.04 LTS / CI Runner) and Windows 11.
- **Toolchain:** CMake (>= 3.20), GCC / Clang / MSVC with C++17 support.
- **Frameworks:** GoogleTest (v1.14+), CTest, lcov/gcov.

7. Test Schedule

- **Phase 1 (Documentation & Test Architecture — Due Sept/Oct 2026):**
- IEEE Software Test Plan definition and sign-off.
- GoogleTest conventions and test directory scaffolding.
- 10 baseline test cases defined across functional, negative, edge, and security scenarios.
- **Sprint 1 (19th Oct – 23rd Oct 2026 — Core Working Product):**
- Automated unit tests for `BitReader` and `BitWriter` boundary handling.
- Unit tests for canonical Huffman tree deserialization and decoding logic.
- Round-trip integration tests (`Compress -> Decompress -> Diff` byte-equality check).
- Minimum 80% line coverage enforcement via automated GitHub Actions CI.
- Validation support for Sprint 1 1-minute working demo video.
- **Sprint 2 (26th Oct – 30th Oct 2026 — Full System & Hardening):**
- Archive statistics calculation unit tests (UC-03).
- CRC-32 integrity validation and corrupted payload detection tests (UC-04).
- Fuzz testing and malformed archive security test cases.
- AddressSanitizer (ASan) and UndefinedBehaviorSanitizer (UBSan) test pipeline.
- Validation support for Sprint 2 2-minute final demo video.

8. Test Deliverables

- Software Test Plan (this document)
- GoogleTest suites in tests/unit/, tests/integration/, and tests/system/

- Automated test execution logs from GitHub Actions CI
- HTML line and branch coverage reports (lcov)
- 40 manual test execution records populated with Actual Results and Pass/Fail status

9. Roles and Responsibilities

Role	Name	Responsibility
Testing & Coverage Lead	Tanmayi	Test Plan architecture, GoogleTest framework, CI coverage monitoring, UC-02 test execution
Architecture & Integration Lead	Saatwik	Core compression tests, round-trip validation, CI integration
CI/CD & Statistics Lead	Yuvraj	GitHub Actions test pipeline, metrics computation test cases
Security & Integrity Lead	Vibhav	CRC-32 tests, corrupted input fuzzing, ASan/UBSan sanitizer checks

10. Risks and Mitigation

- **Risk:** Mismatch between bit-writing order and bit-reading order breaking decompression.
  Mitigation: Formal sign-off on the MSB-first contract in the Binary Archive Specification before Sprint 2 implementation.
- **Risk:** Incomplete test coverage on edge cases.
  Mitigation: CI gated checks enforcing 80% line coverage threshold.

11. Assumptions & Dependencies

- Standard C++17 library support on target execution environments.
- Continuous integration runners have access to CMake and GoogleTest packages.

12. Suspension & Resumption Criteria

- **Suspension:** CI build failures or severe memory faults crashing the test runner.
- **Resumption:** Fixing the breaking commit on the local feature branch prior to merging.

13. Test Case Specifications (Baseline Test Suite)

In accordance with course deliverable requirements, the baseline test suite specifies 10 concrete test cases covering positive, negative, edge, performance, and security scenarios.

13.1 Test Case Summary Table

Test ID	Category	Target Module / Requirement	Description	Expected Outcome
TC-COMP-01	Positive	Core Compression (`JACK-F-001` , `JACK-F-004`)	Compress ASCII text file with repeated characters	Valid `.jack` archive produced; header matches spec
TC-COMP-02	Edge Case	Compression Core (`JACK-F-002` , `JACK-F-003`)	Compress single-character repeated file (`"AAAAA..."`)	Single-node tree handled cleanly; valid archive
TC-COMP-03	Edge Case	Compression Core (`JACK-NF-003`)	Compress 0-byte empty file	Archive created with zero payload or graceful empty notice
TC-DECOMP-01	Positive	Decompression Core (`JACK-F-008`)	Full round-trip decompress of valid `.jack` archive	Restored file is 100% byte-identical (`diff` / SHA-256 match)
TC-DECOMP-02	Negative	Decompression Core (`JACK-F-005` , `JACK-SR-001`)	Decompress corrupted archive with invalid magic bytes	Fails immediately with descriptive error; no crash
TC-DECOMP-03	Edge Case	BitReader (`JACK-F-007`)	BitReader boundary test on unaligned bit count (e.g., 13 bits)	Reads exactly 13 bits without consuming extra trailing byte padding
TC-STAT-01	Positive	Statistics Engine (`JACK-F-009` , `JACK-F-010`)	Inspect valid archive statistics via `-s` / `--stat`	Displays correct original size, compressed size, and ratio
TC-VALID-01	Negative	Integrity Validation (`JACK-F-014` , `JACK-SR-002`)	Validate archive with 1-bit payload corruption via `-t`	CRC-32 mismatch detected; returns exit code 3
TC-PERF-01	Performance	System Performance (`JACK-NF-001`)	Compress and decompress 5 MB sample text/binary	Execution time < 1.0 second; peak RAM < 32 MB
TC-SEC-01	Security	Archive Parsing (`JACK-SR-003` , `JACK-SR-005`)	Decompress archive with truncated payload (unexpected EOF)	Graceful error; zero buffer overrun / memory leak (ASan clean)

13.2 Detailed Test Case Specifications

TC-COMP-01: Standard ASCII File Compression (Positive)

- **Requirement ID:** `JACK-F-001` , `JACK-F-004`
- **Preconditions:** Valid text file `sample.txt` containing typical English prose exists.
- **Test Steps:**
- Execute `jackfruit compress sample.txt -o sample.txt.jack` .
- Inspect exit code.
- Inspect output file header bytes using hex viewer / `xxd` .
- **Expected Result:**
- Process exits with code 0.
- `sample.txt.jack` starts with magic bytes `0x4A 0x41 0x43 0x4B` (`"JACK"`).
- Compressed file size is smaller than original size for redundant text.

TC-COMP-02: Single Distinct Character File (Edge Case)

- **Requirement ID:** `JACK-F-002`, `JACK-F-003`
- **Preconditions:** A file `single.txt` containing 1,000 occurrences of character `'A'` and no other symbols.
- **Test Steps:**
- Execute `jackfruit compress single.txt -o single.txt.jack`.
- Decompress archive using `jackfruit decompress single.txt.jack -o single_out.txt`.
- **Expected Result:**
- Tree builder correctly handles degenerate single-symbol tree (depth 1 or explicit representation).
- `single_out.txt` is byte-identical to `single.txt`.

TC-COMP-03: Zero-Byte Empty File (Edge Case)

- **Requirement ID:** `JACK-NF-003`
- **Preconditions:** File `empty.txt` with size 0 bytes.
- **Test Steps:**
- Execute `jackfruit compress empty.txt -o empty.txt.jack`.
- Execute `jackfruit decompress empty.txt.jack -o empty_out.txt`.
- **Expected Result:**
- Program does not crash or throw division-by-zero on ratio calculations.
- Restored file `empty_out.txt` has size 0 bytes.

TC-DECOMP-01: Round-Trip Restoration Verification (Positive)

- **Requirement ID:** `JACK-F-008`
- **Preconditions:** `large_code.cpp` exists in test fixtures.
- **Test Steps:**
- Run `jackfruit compress large_code.cpp -o compressed.jack`.
- Run `jackfruit decompress compressed.jack -o restored.cpp`.
- Compare `large_code.cpp` and `restored.cpp` using `cmp` / `sha256sum`.
- **Expected Result:**
- Exit code 0 on both commands.
- Hashes match exactly with zero difference.

TC-DECOMP-02: Header Magic Byte Corruption (Negative)

- **Requirement ID:** `JACK-F-005`, `JACK-SR-001`
- **Preconditions:** A valid archive `valid.jack` with the first byte modified from `0x4A` to `0xFF`.
- **Test Steps:**
- Execute `jackfruit decompress corrupted.jack -o output.bin`.
- **Expected Result:**
- Program immediately aborts decompression with exit code 2.
- Outputs clear stderr message: `Error: Invalid archive format (magic bytes mismatch)`.
- No partial or corrupted file is written to output location.

TC-DECOMP-03: BitReader Non-Byte-Aligned Boundary Reading (Edge Case)

- **Requirement ID:** `JACK-F-007`
- **Preconditions:** Bit stream containing exactly 13 encoded bits (`0b11010011 0b10100000`).
- **Test Steps:**
- Instantiate `BitReader` with buffer and `total_bits = 13`.

- Read 13 single bits sequentially.
- Attempt to read the 14th bit.
- **Expected Result:**
- Exactly 13 bits are read with correct bit sequence.
- 14th read returns EOF (`false` or throws `EndOfStreamException`).
- The remaining 3 padding bits in byte 2 are ignored.

TC-STAT-01: Archive Metadata and Statistics Extraction (Positive)

- **Requirement ID:** `JACK-F-009` , `JACK-F-010` , `JACK-F-012`
- **Preconditions:** Pre-generated archive `test.jack` with original size = 10,000 bytes, compressed size = 6,000 bytes.
- **Test Steps:**
- Execute `jackfruit stat test.jack` .
- **Expected Result:**
- Output displays:
  - Original Size: 10,000 bytes
  - Compressed Size: 6,000 bytes
  - Compression Ratio: 40.0% (or 0.60x)
  - CRC-32 Checksum in hexadecimal format
- Exit code 0.

TC-VALID-01: Corrupted Payload CRC-32 Detection (Negative)

- **Requirement ID:** `JACK-F-014` , `JACK-SR-002`
- **Preconditions:** Pre-generated archive where byte 25 of the compressed bitstream is flipped from `0x00` to `0x01` .
- **Test Steps:**
- Execute `jackfruit validate test_flipped.jack` .
- **Expected Result:**
- Program recalculates payload CRC-32 and compares against stored header checksum.
- Displays: `Integrity check FAILED: Checksum mismatch (Expected: 0x..., Computed: 0x...)` .
- Exit code 3.

TC-PERF-01: Throughput and Sub-Second Execution (Performance)

- **Requirement ID:** `JACK-NF-001`
- **Preconditions:** Benchmark text file `5mb_dataset.txt` (5,242,880 bytes).
- **Test Steps:**
- Time execution of `jackfruit compress 5mb_dataset.txt -o out.jack` .
- Time execution of `jackfruit decompress out.jack -o out.txt` .
- **Expected Result:**
- Compression completes in < 1.0 second on standard reference test hardware.
- Decompression completes in < 0.6 seconds.
- Peak memory footprint remains < 32 MB.

TC-SEC-01: Truncated Payload / Unexpected EOF Protection (Security)

- **Requirement ID:** `JACK-SR-003`, `JACK-SR-005`
- **Preconditions:** Archive header claims 10,000 payload bits, but file ends prematurely after 12 bytes.
- **Test Steps:**
- Run `jackfruit decompress truncated.jack -o restored.bin` under AddressSanitizer (`-fsanitize=address`).
- **Expected Result:**
- Program terminates gracefully with `Error: Unexpected EOF while reading bitstream`.
- AddressSanitizer reports 0 heap-buffer-overflows and 0 memory leaks.

14. Test Metrics & Reporting

- Percentage of test cases executed and passed.
- Line coverage percentage (target >= 80%) and branch coverage (target >= 70%).
- Defect density categorized by module.

15. Approvals

Role	Name	Signature / Date
Testing Lead	Tanmayi	Approved — Sprint 1
Project Lead	Saatwik	Approved — Sprint 1